

Test 3 - OOP - QT designer

(fields separated by ";" in text file)

```
void Repository::loadFromFile() {
    std::ifstream fm(filename);
    std::string line;
    while (getline(fm, line)) {
        std::stringstream ss(line);
        std::string field1;
        std::string field2;
        :
        std::getline(ss, field1, ';');
        std::getline(ss, field2, ';');
        :
        double field3 = stod(
        double value1 = stod(field1);
        int value2 = stoi(field2);
        bool value3 = false;
        if (field3.find("true") != std::string::npos) {
            value3 = true;
        }
        bills.emplace_back(field1, field2, ...);
    }
    if (!fm.is_open()) {
        throw std::runtime_error(...);
    }
    fm.close();
}
```

```
int main(int argc, char *argv[]) {
    QApplication app(argc, argv);
    Repository repo{"filename.txt"};
    class name → Gui gui{repo}; → obj name
    gui.show();
    return app.exec();
}
```

Select item → sth else happens:

```
void Gui::populate...() {
    ui->vegetablesListWidget->clear();
    auto sel = ui->familyListWidget->selectedItem();
    std::string family
```

Search in repo for sth:

```
for (...) {
    if (...)
        return;
}
throw std::runtime_error("not f.")
```

When to CLEAR ui elements:

- before repopulating a list
- after successful action
- if try-catch catches an error

```
Gui::Gui(Repository & repo, QWidget *parent) : QWidget(parent), ui(new Ui::Gui), repo{repo} {
    ui->setupUi(this);
    populateList();
    connectSignalsAndSlots();
}
```

for (auto e: elems) → copy → changes to e won't affect vector
 for (auto& e: elems) → modify original (good for update)
 for (const auto& e: elems) → no copy, no modify (good for read, print, filter)

```
void Gui::populateList() {
    ui->billsListWidget->clear();
    auto bills = repo.getAll();
    for (const auto& b: bills) {
        auto item = new QListWidgetItem(QString::fromStdString(b.toString()));
        ui->billsListWidget->addItem(item);
    }
    ...
}
```

```
void Gui::connectSignalsAndSlots() {
    connect(ui->buttonName, &QPushButton::clicked, this, &Gui::functionName)
    ...
}
```

std::set < std::string >
 ↳ ensures uniqueness
 ↳ sorts automatically

SportsTest300P

```
std::pair< std::vector< std::string>, int> getIntervalsAndHours(..., ...) {  
    std::vector< std::string> intervals;  
    int total = 0;  
    return {intervals, total};  
}
```

=> UNPACK tuples: auto result = repo.getIntervalsAndHours(...);
auto intervals = result.first;
auto hours = result.second;

Take information from a QLineEdit:

```
std::string intensity = ui->intensityLineEdit->text().toStdString();  
int number = ui->lineEdit->text().toInt();
```

Put information in a QListWidget:

```
for(...) {  
    auto item = new QListWidgetItem(QString::fromStdString(...));  
    ui->listWidget->addItem(item);  
}
```

→ faster: ui->listWidget->addItem(QString::fromStdString(...));

Put information on a Label: ui->solLabel->setText(QString::fromStdString(sol));
ui->label->setText(QString::number(...));

Catch exceptions w message boxes:

```
try {  
    ...  
} catch (const std::exception& e) {  
    QMessageBox::critical(this, "ERROR", e.what());  
}
```

connect signals and slots:

• BUTTON: connect(ui->filterButton, &QPushButton::clicked, this, &Gui::filter)

• LINEEDIT: connect(ui->searchLineEdit, &QLineEdit::textChanged, this, &Gui::show)
(dynamic search)

Sort by double criterion:

```
std::sort(items.begin(), items.end(), [](const Item& a, const Item& b) {  
    if (a.getCategory() != b.getCategory()) {  
        return a.getCategory() < b.getCategory();  
    }  
    return a.getName() < b.getName();  
});  
return items;
```

Dynamic filter (substrings, while typing)

```
std::vector<Item> result;  
for(const auto& i: items) {  
    if (i.getCategory().find(input) != std::string::npos ||  
        i.getName().find(input) != std::string::npos) {  
        result.push_back(i);  
    }  
}
```

Set background color of an item in a list, Set same color as object:

```
item->setBackground(Qt::green)
```


Message Boxes:

QMessageBox::information(this, "Success", "Successful addition."); ✓

OR

```
QMessageBox msgBox(this);  
msgBox.setText("Successful addition");  
msgBox.exec();
```

message box with function:

```
QMessageBox::information(this, "best", QString::from  
fromStdString(display_text)); // if it's a string  
QString::number(display_text); // if it's a nr.
```

Compute an equation when selected based by indexed vector:

→ select item:

```
auto selection = ui->listWidget->selectedItems(); => LIST  
if(selection.empty()) return;
```

→ retrieve the text from that item

```
int index =  
std::string equation = selection[0]->text().toStdString();
```

~~int index =~~

→ retrieve row number of the selected item

```
int index = ui->listWidget->currentRow();
```

if no sorting/checked boxes.

→ return from repo that equation stored at that position in the memory

```
Equation eq = repo.getEq(index);
```

```
std::string sol = repo.solution(...)
```

Add template:

```
for(...) {  
    if(...) {  
        throw std::runtime_error("Already exists");  
    }  
    auto eq = Equation(...); // create obj.  
    equations.push-back(eq);  
}
```

get to print less fields than the domain has: (no toString)

```
std::string text = d.getName() + '!' + d.getKeywords();
```

```
auto item = new QListWidgetItem(QString::fromStdString(text));
```

Retrieve only unique-field elements (if I have

1. Family Mia | Hello) I want only
2. Family Mia | Fine since the
Family Mia.

```
std::vector<std::string> Repo::getFamilies() const {
```

```
std::vector<std::string> families;
```

```
for(const auto& v: vegetables) {
```

```
std::string family = v.getFamily();
```

```
if (std::find(families.begin(), families.end(), family) == families.end()) {
```

```
    families.push-back(family);
```

```
} }
```

```
std::sort(families.begin(), families.end());
```

```
return families;
```


Highlight object in a list

```
item → text() → item → setBackground(QBrush(QColor("yellow")))
std::string name = ui → veggieLineEdit → text().toStdString();
try {
    Vegetable v = repo.getVegetableByName(name);
    ui → label → setText(QString::fromStdString(v.getParts()));
    for (int i = 0; i < ui → familyListWidget → count(); i++) {
        QListWidgetItem* = ui → familyListWidget → item(i);
        if (item → text().toStdString() == v.getFamily()) {
            item → setBackground(QBrush(QColor("yellow")));
        }
        else {
            item → setBackground(QBrush());
        }
    }
    ui → familyListWidget → clearSelection();
    ui → familyListWidget → viewport() → update(); // show highlight without delay.
}
catch (std::exception & e) {
    QMessageBox::critical(this, "ERROR", e.what());
}
```

Checkboxes

```
auto bills = repo.getAll();
bool unpaid = ui → unpaidCheckBox → isChecked();
bool paid = ui → paidCheckBox → isChecked();
for (const auto & bill : bills) {
    if (paid && !unpaid && !bill.getIsPaid()) {
        continue;
    }
    if (!paid && unpaid && bill.getIsPaid()) {
        continue;
    }
    std::string text = ...
    auto item = new QListWidgetItem(QString::fromStdString(text));
    if (!bill.getIsPaid()) {
        item → setBackground(QBrush(QColor("red")));
    }
    ui → billsListWidget → addItem(item);
}
```

Exact match vs filter:

Exact match:

```
for (...) {
    if (c.getName() == name)
        ...
}
```

Filter (partial match):

```
for (...) {
    if (c.getName().find(name) != std::string::npos)
        ...
}
```


cars By Manufacturer (repo style)

```
std::vector<Car> result; ... (name)
for (const ... ) {
    if (c.getName().find(name) != std::string::npos) {
        ...
    }
}
if (result.empty()) {
    throw std::runtime_error("No. found");
}
return result;
```

populate list with vector of strings

```
ui → triangleListWidget → clear();
auto triangles = repo.getAll();
auto strings = repo.getInMathematicalForm();
for (size_t i = 0; i < triangles.size(); i++) {
    auto * item = new QListWidgetItem(QString::fromStdString(strings[i]));
    if (triangles[i].area() > 10)
        item → setBackground(Qt::green);
    ui → triangleListWidget → addItem(item);
}
```

Change font:

```
QFont font = item → font();
```

```
font.setBold(true);
item → setFont(font);
```

Change background:

```
item → setBackground(Qt::green);
```

```
void paintEvent(QPaintEvent* event) {
    QPainter painter(this);
    int startX = 220;
    int y = 40;
    for (...) {
        int radius = ...
        painter.drawText(20, y, QString::fromStdString(p.get...()));
        painter.drawEllipse(startX - radius, y - radius, 2 * radius, 2 * radius);
        painter.drawRect(startX - radius, y - radius, 2 * radius, 2 * radius);
        painter.drawPoint(startX, y);
        painter.drawLine(startX, y, startX + 100, y);
    }
    y += 80;
```

QPainter

```
class Statistics : public QWidget, public Observer {
    Q_OBJECT
```

public:

```
explicit Statistics(Service &service, QWidget* parent = nullptr);
```

~

```
void update() override;
```

```
repaint()
```

protected:

```
void paintEvent(QPaintEvent* event) override;
```

private:

```
painter.drawText(x, y, QString::fromStdString(s));
```

```
QPainter painter(this);
```

```
for (...) {
```

```
    painter.setBrush(Qt::red);
```

```
    int x, y, radius;
```

```
    painter.drawEllipse(QPoint(x, y), radius, radius);
```



```
class StarTableModel: public QAbstractTableModel {
```

Q_OBJECT

private:

```
std::vector<Star> stars;
```

public:

```
explicit StarTableModel (const std::vector<Star>& stars,  
                          QObject *parent = nullptr);
```

1) int rowCount (...);

2) int columnCount (...);

3) QVariant data (...);

QVariant headerData (...) ...;

4) void updateData (const std::vector<Star>& newData);
Qt::ItemFlags flags (const QModelIndex& index) const override;

```
};
```

```
StarTableModel::StarTableModel (const std::vector<Star> &stars, QObject *parent):
```

```
stars { stars }, QAbstractTableModel { parent } {}
```

```
... rowCount ... {
```

```
return static_cast<int> (stars.size());
```

```
}
```

```
... columnCount ... {
```

```
return 5;
```

```
}
```

```
... data (...) ... {
```

```
if (!index.isValid() || role != Qt::DisplayRole)
```

```
return QVariant {};
```

```
const auto& s = stars[index.row()];
```

```
if (index.column() == 0)
```

```
return QString::fromStdString (s.getStarName());
```

```
...
```

```
if (index.column() == 4)
```

```
return s.getDiameter();
```

```
return QVariant {};
```

```
}
```

```
... headerData ...
```

```
... updateData ... {
```

```
beginResetModel(); stars = newData; endResetModel();
```

```
};
```

GVI.h:

StarTableModel *model

GVI.cpp:

model = new StarTableModel;

service.getStarsSorted(), this?

ui->tableView->setModel(model)